

2.2 - What Is Algorithm Analysis?

It is very common for beginning computer science students to compare their programs with one another. You may also have noticed that it is common for computer programs to look very similar, especially the simple ones. An interesting question often arises. When two programs solve the same problem but look different, is one program better than the other?

In order to answer this question, we need to remember that there is an important difference between a program and the underlying algorithm that the program is representing. As we stated in Chapter 1, an algorithm is a generic, step-by-step list of instructions for solving a problem. It is a method for solving any instance of the problem so that given a particular input, the algorithm produces the desired result. A program, on the other hand, is an algorithm that has been encoded into some programming language. There may be many programs for the same algorithm, depending on the programmer and the programming language being used.

To explore this difference further, consider the function shown in [Listing 2.2.1](#). This function solves a familiar problem, computing the sum of the first n integers. The algorithm uses the idea of an accumulator variable that is initialized to 0. The solution then iterates through the n integers, adding each to the accumulator.

Listing 2.2.1.

```
public class FindSum {  
    public static long sumOfN(int n) {  
        long theSum = 0;  
        for (int i = 1; i <= n; i++) {  
            theSum = theSum + i;  
        }  
        return theSum;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(sumOfN(10));  
    }  
}
```

► Aside

Now look at the method in [Listing 2.2.2](#). At first glance it may look strange, but upon further inspection you can see that this function is essentially doing the same thing as the previous one. The reason this is not obvious is poor coding. We did not use good identifier names to assist with readability, and we used an extra assignment statement that was not really necessary during the accumulation step.

Listing 2.2.2.

```
public class FindSum2 {  
    public static long foo(int tom) {  
        long fred = 0;  
        for (int nancy = 1; nancy <= tom; nancy++) {  
            long joanne = nancy;  
            fred = fred + joanne;  
        }  
        return fred;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(foo(10));  
    }  
}
```

The question we raised earlier asked whether one method is better than another. The answer depends on your criteria. The method `sumOfN` is certainly better than the method `foo` if you are concerned with readability. In fact, you have probably seen many examples of this in your introductory programming course since one of the goals there is to help you write programs that are easy to read and easy to understand. In this course, however, we are also interested in characterizing the algorithm itself. (We certainly hope that you will continue to strive to write readable, understandable code.)

Algorithm analysis is concerned with comparing algorithms based upon the amount of computing resources that each algorithm uses. We want to be able to consider two algorithms and say that one is better than the other because it is more efficient in its use of those resources or perhaps because it simply uses fewer. From this perspective, the two methods above seem very similar. They both use essentially the same algorithm to solve the summation problem.

At this point, it is important to think more about what we really mean by computing resources. There are two different ways to look at this. One way is to

consider the amount of space or memory an algorithm requires to solve the problem. The amount of space required by a problem solution is typically dictated by the problem instance itself. Every so often, however, there are algorithms that have very specific space requirements, and in those cases we will be very careful to explain the variations.

As an alternative to space requirements, we can analyze and compare algorithms based on the amount of time they require to execute. This measure is sometimes referred to as the *execution time* or *running time* of the algorithm. One way we can measure the execution time for the function `sumOfN` is to do a **benchmark analysis**. This means that we will track the actual time required for the program to compute its result. In Java, we can benchmark a function by noting the starting time and ending time within the system we are using. In the `System` module there is a method called `nanoTime` that will return the amount of time that the Java virtual machine has been running, in nanoseconds. By calling this function twice, at the beginning and at the end, and then computing the difference, we can get the number of seconds (fractions in most cases) for execution.

[Listing 2.2.3](#) lets you enter the number you want to sum up to (`n`). It then invokes the `sumOfN` method 25 times and calculates the time required for each trial:

Listing 2.2.3.

```

1  import java.util.Scanner;
2
3  public class Timing {
4      public static long sumOfN(long n) {
5          long theSum = 0;
6          for (int i = 1; i <= n; i++) {
7              theSum = theSum + i;
8          }
9          return theSum;
10     }
11
12     public static void main(String[] args) {
13
14         Scanner input = new Scanner(System.in);
15         System.out.print("Find sum from 1 to n: ");
16         long n = input.nextInt();
17
18         for (int trial = 0; trial < 25; trial++) {
19             long startTime = System.nanoTime();
20             long result = sumOfN(n);
21
22             double elapsed = (System.nanoTime() - startTime) / 1.0E9;
23             System.out.printf("Trial %d: Sum %d: time %.6f sec.%n",
24                             trial, result, elapsed);
25         }
26     }
27 }

```

[Listing 2.2.3](#) shows the original `sumOfN` function with the timing calls embedded before and after the summation. The function returns a tuple consisting of the result and the amount of time (in seconds) required for the calculation. Here is start and end of the output when we enter 10000 for the limit of the sum:

```

Trial 0: Sum 50005000: time 0.003886 sec.
Trial 1: Sum 50005000: time 0.004009 sec.
Trial 2: Sum 50005000: time 0.000186 sec.
Trial 3: Sum 50005000: time 0.000185 sec.
...
Trial 20: Sum 50005000: time 0.000125 sec.
Trial 21: Sum 50005000: time 0.000124 sec.
Trial 22: Sum 50005000: time 0.000125 sec.
Trial 23: Sum 50005000: time 0.000124 sec.
Trial 24: Sum 50005000: time 0.000124 sec.

```

Why does the time go down from .003886 seconds to .000124? Because the Java Virtual machine and the computer hardware itself **cache** results, keeping them in memory for future access. We run the trial loop 25 times in order to give the cache time to stabilize.

We discover that the time is fairly consistent and it takes on average about 0.000125 seconds to execute that code. What if we run the function adding the first 100,000 integers? (Showing only the final five trials)

```
Trial 20: Sum 5000050000: time 0.001225 sec.  
Trial 21: Sum 5000050000: time 0.001226 sec.  
Trial 22: Sum 5000050000: time 0.001225 sec.  
Trial 23: Sum 5000050000: time 0.001224 sec.  
Trial 24: Sum 5000050000: time 0.001224 sec.
```

Again, the time required for each run, although longer, is very consistent, averaging about 10 times more seconds. For n equal to 1,000,000 we get:

```
Trial 20: Sum 500000500000: time 0.012350 sec.  
Trial 21: Sum 500000500000: time 0.012411 sec.  
Trial 22: Sum 500000500000: time 0.012353 sec.  
Trial 23: Sum 500000500000: time 0.012443 sec.  
Trial 24: Sum 500000500000: time 0.012447 sec.
```

In this case, the average again turns out to be about 10 times the previous experiment.

Now consider [Listing 2.2.4](#), which shows a different means of solving the summation problem. This method, `sumOfNImproved`, takes advantage of a closed equation $\sum_{i=1}^n i = \frac{(n)(n+1)}{2}$ to compute the sum of the first n integers without iterating.

Listing 2.2.4.

```
public static long sumOfNImproved(long n) {  
    long theSum = n * (n + 1) / 2;  
    return theSum;  
}
```

We then change the call in line 23 of [Listing 2.2.3](#) to:

```
long result = sumOfNImproved(n);
```

If we do the same benchmark measurement with this revised code, using five different values for n (10,000, 100,000, 1,000,000, 10,000,000, and 100,000,000), we get the following results from averaging the last five trials:

```
Sum 50005000:      time 0.0000088 sec.  
Sum 5000050000:    time 0.0000092 sec.  
Sum 500000500000:  time 0.0000082 sec.  
Sum 5000000500000: time 0.0000078 sec.
```

There are two important things to notice about this output. First, the times recorded above are shorter than any of the previous examples. Second, they are very consistent no matter what the value of n . It appears that `sumOfNImproved` is hardly impacted by the number of integers being added.

But what does this benchmark really tell us? Intuitively, we can see that the iterative solutions seem to be doing more work since some program steps are being repeated. This is likely the reason it is taking longer. Also, the time required for the iterative solution seems to increase as we increase the value of n . However, if we ran the same function on a different computer or used a different method language, we would likely get different results. It could take even longer to perform `sumOfNImproved` if the computer were older.

We need a better way to characterize these algorithms with respect to execution time. The benchmark technique computes the actual time to execute. It does not really provide us with a useful measurement because it is dependent on a particular machine, program, time of day, compiler, and programming language. Instead, we would like to have a characterization that is independent of the program or computer being used. This measure would then be useful for judging the algorithm alone and could be used to compare algorithms across implementations.

You have attempted 1 of 1 activities on this page.



Building agents? Free virtual conference, Oct 22. 122 speakers, zero vendor pitches. **Register today**

www.alldayai.com

Ads by EthicalAds

2.3 - Big O Notation

When trying to characterize an algorithm's efficiency in terms of execution time, independent of any particular program or computer, it is important to quantify the number of operations or steps that the algorithm will require. If each of these steps is considered to be a basic unit of computation, then the execution time for an algorithm can be expressed as the number of steps required to solve the problem. Deciding on an appropriate basic unit of computation can be a complicated problem and will depend on how the algorithm is implemented.

A good basic unit of computation for comparing the summation algorithms shown earlier might be the number of assignment statements performed to compute the sum. In the function `sumOfN`, the number of assignment statements is 1 (*theSum* = 0) plus the value of *n* (the number of times we perform *theSum* = *theSum* + *i*). We can denote this by a function, call it *T*, where $T(n) = 1 + n$. The parameter *n* is often referred to as the "size of the problem," and we can read this as "*T*(*n*) is the time it takes to solve a problem of size *n*, namely $1 + n$ steps."

In the summation functions given above, it makes sense to use the number of terms in the summation to denote the size of the problem. We can then say that the sum of the first 100,000 integers is a bigger instance of the summation problem than the sum of the first 1,000. Because of this, it might seem reasonable that the time required to solve the larger case would be greater than for the smaller case. Our goal then is to show how the algorithm's execution time changes with respect to the size of the problem.

Computer scientists prefer to take this analysis technique one step further. It turns out that the exact number of operations is not as important as determining the most dominant part of the $T(n)$ function. In other words, as the problem gets larger, some portion of the $T(n)$ function tends to overpower the rest. This dominant term is what, in the end, is used for comparison. The **order of magnitude** function describes the part of $T(n)$ that increases the fastest as the value of *n* increases. Order of magnitude is often called **Big O notation** (O stands for *order*) and written as $O(f(n))$. It provides a useful approximation of the actual number of steps in the computation. The function $f(n)$ provides a simple representation of the dominant part of the original $T(n)$.

In the above example, $T(n) = 1 + n$. As n gets larger, the constant 1 will become less and less significant to the final result. If we are looking for an approximation for $T(n)$, then we can drop the 1 and simply say that the running time is $O(n)$. It is important to note that the 1 is certainly significant for $T(n)$. However, as n gets large, our approximation will be just as accurate without it.

As another example, suppose that for some algorithm, the exact number of steps is $T(n) = 5n^2 + 27n + 1005$. When n is small, say 1 or 2, the constant 1005 seems to be the dominant part of the function. However, as n gets larger, the n^2 term becomes the most important. In fact, when n is really large, the other two terms become insignificant in the role that they play in determining the final result. Again, to approximate $T(n)$ as n gets large, we can ignore the other terms and focus on $5n^2$. In addition, the coefficient 5 becomes insignificant as n gets large. We would say then that the function $T(n)$ has an order of magnitude $f(n) = n^2$, or simply that it is $O(n^2)$.

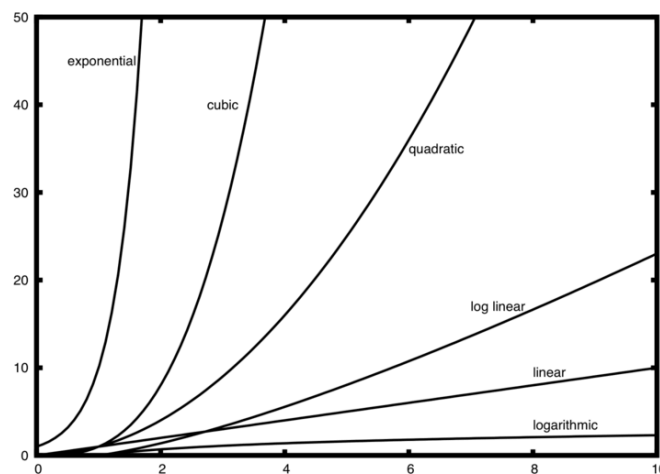
Although we do not see this in the summation example, sometimes the performance of an algorithm depends on the exact values of the data rather than simply the size of the problem. For these kinds of algorithms we need to characterize their performance in terms of **best-case**, **worst-case**, or **average-case** performance. The worst-case performance refers to a particular data set where the algorithm performs especially poorly, whereas a different data set for the exact same algorithm might have extraordinarily good (best-case) performance. However, in most cases the algorithm performs somewhere in between these two extremes (average-case performance). It is important for a computer scientist to understand these distinctions so they are not misled by one particular case.

A number of very common order of magnitude functions will come up over and over as you study algorithms. These are shown in [Table 2.3.1](#). In order to decide which of these functions is the dominant part of any $T(n)$ function, we must see how they compare with one another as n gets large.

Table 2.3.1. Common Functions for Big O

$f(n)$	Name
1	<i>Constant</i>
$\log n$	<i>Logarithmic</i>
n	<i>Linear</i>
$n \log n$	<i>Log linear</i>
n^2	<i>Quadratic</i>
n^3	<i>Cubic</i>
2^n	<i>Exponential</i>

[Figure 2.3.2](#) shows graphs of the common functions from [Table 2.3.1](#). Notice that when n is small, the functions are not very well defined with respect to one another. It is hard to tell which is dominant. However, as n grows, there is a definite relationship and it is easy to see how they compare with one another.

**Figure 2.3.2.** Plot of Common Big O Functions

As a final example, suppose that we have the fragment of Java code shown in [Listing 2.3.3](#). Although this program does not really do anything, it is instructive to see how we can take actual code and analyze performance.

Listing 2.3.3.

```
int a = 5;
int b = 6;
int c = 10;
int n = 1000;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        int x = i * i;
        int y = j * j;
        int z = i * j;
    }
}

for (int k = 0; k < n; k++) {
    int w = a * k + 45;
    int v = b * b;
}
int d = 33;
```

The number of assignment operations is the sum of four terms. The first term is the constant 4, representing the four assignment statements at the start of the fragment. The second term is $3n^2$, since there are three statements that are performed n^2 times due to the nested iteration. The third term is $2n$, two statements iterated n times. Finally, the fourth term is the constant 1, representing the final assignment statement. This gives us $T(n) = 4 + 3n^2 + 2n + 1 = 3n^2 + 2n + 5$. By looking at the exponents, we can see that the n^2 term will be dominant and therefore this fragment of code is $O(n^2)$. Note that all of the other terms as well as the coefficient on the dominant term can be ignored as n grows larger.

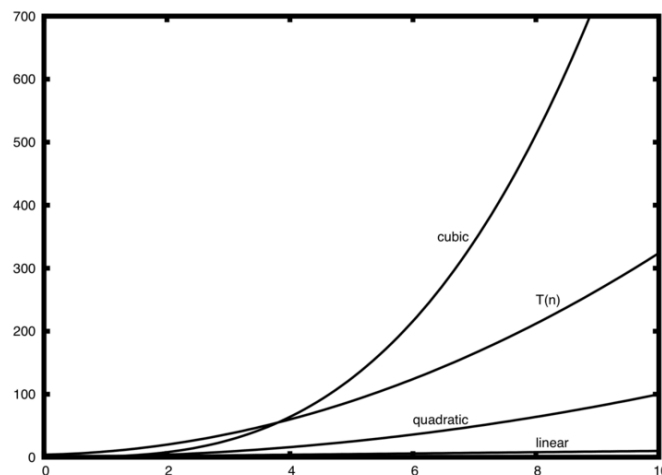


Figure 2.3.4. Comparing $T(n)$ with Common Big O Functions

Figure 2.3.4 shows a few of the common Big O functions as they compare with the $T(n)$ function discussed above. Note that $T(n)$ is initially larger than the cubic function. However, as n grows, the cubic function quickly overtakes $T(n)$. We can also see that $T(n)$ then follows the quadratic function as n continues to grow.

Self Check

Write two Java methods to find the minimum number in a list. The first function should compare each number to every other number on the list. $O(n^2)$. The second function should be linear $O(n)$.

You have attempted 1 of 1 activities on this page.



Bruno
The Git-native
API client

Git-native API client
Send requests to REST,
GraphQL and gRPC.
Collections live in git.
Get Bruno
usebruno.com

Ads by EthicalAds

2.4 - An Anagram Detection Example

A good example problem for showing algorithms with different orders of magnitude is the classic anagram detection problem for strings. One string is an ***anagram*** of another if the second is simply a rearrangement of the first. For example, heart and earth are anagrams. The strings taster and treats are anagrams as well. For the sake of simplicity, we will assume that the two strings in question are of equal length and that they are made up of symbols from the set of 26 lowercase alphabetic characters. Our goal is to write a boolean function that will take two strings and return whether they are anagrams.

2.4.1 - Solution 1: Anagram Detection Checking Off

Our first solution to the anagram problem will check the lengths of the strings and then check to see that each character in the first string actually occurs in the second. If it is possible to check off each character, then the two strings must be anagrams. ***Checking off*** a character will be accomplished by setting an element to - if it is found (it's not a letter, so it won't ever match letters in the first string).

However, since strings in Java are immutable, the first step in the process will be to convert the second string to an array of characters with the data type `char`. The `char` data type stores exactly one character, and we can use `==` to compare them directly (unlike `Strings`).

We will get each character from the first string by using the `charAt()` method and check it against the characters in the array formed from the second string. If a letter is found, we check it off by setting the array element to -. [Listing 2.4.1](#) shows this method.

Listing 2.4.1.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 public class Anagrams1 {
2
3     public static boolean anagramSolution1(String s1, String
4         boolean isAnagram = true;
5
6     if (s1.length() != s2.length()) {
7         isAnagram = false;
8     } else {
9         int pos1 = 0;
10        char[] s2Array = s2.toCharArray();
11        while (pos1 < s1.length() && isAnagram) {
12            int pos2 = 0;
13            boolean found = false;
14            while (pos2 < s2.length() && !found) {
15                if (s1.charAt(pos1) == s2Array[pos2]) {
16                    found = true;
17                } else {
18                    pos2 = pos2 + 1;
19                }
20            }
21            if (found) {
22                s2Array[pos2] = '-';
23            } else {
24                isAnagram = false;
25            }
26            pos1 = pos1 + 1;
27        }
28    }
29    return isAnagram;
30 }
```

To analyze this algorithm, we need to note that each of the n characters in s_1 will cause an iteration through up to n characters in the list from s_2 . Each of the n positions in the s_2 array will be visited once to match a character from s_1 . The number of visits then becomes the sum of the integers from 1 to n . We stated earlier that this can be written as

$$\begin{aligned}\sum_{i=1}^n i &= \frac{n(n+1)}{2} \\ &= \frac{1}{2}n^2 + \frac{1}{2}n\end{aligned}$$

As n gets large, the n^2 term will dominate the n term and the $\frac{1}{2}$ can be ignored. Therefore, this solution is $O(n^2)$.

This is a worst-case analysis, and it happens when the strings are anagrams—the algorithm has to compare every pair of characters. If the words are not anagrams, the algorithm will exit before all the comparisons are done (as soon as a mismatch is found).

2.4.2 - Anagram Detection Solution 2: Sort and Compare

Another solution to the anagram problem will make use of the fact that even though s_1 and s_2 are different, they are anagrams only if they consist of exactly the same characters. So if we begin by sorting each string alphabetically from a to z, we will end up with the same string if the original two strings are anagrams. [Listing 2.4.2](#) shows this solution. Again, in Java we can use a `sort` method on arrays by converting each string to an array at the start.

Listing 2.4.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Arrays; // note the "s" at the end!
2
3 public class Anagrams2 {
4
5     public static boolean anagramSolution2(String s1, String
6         char[] s1Array = s1.toCharArray();
7         char[] s2Array = s2.toCharArray();
8
9         Arrays.sort(s1Array);
10        Arrays.sort(s2Array);
11
12        int pos = 0;
13        boolean matches = true;
14
15        while (pos < s1.length() && matches) {
16            if (s1Array[pos] == s2Array[pos]) {
17                pos = pos + 1;
18            } else {
19                matches = false;
20            }
21        }
22        return matches;
23    }
24
25    public static void main(String[] args) {
26        System.out.println(anagramSolution2("taster", "treat");
27        System.out.println(anagramSolution2("abcd", "dcab"));
28        System.out.println(anagramSolution2("abcd", "dcda"));
29    }
30 }
```

At first glance you may be tempted to think that this algorithm is $O(n)$, since there is one simple iteration to compare the n characters after the sorting process. However, the two calls to the Java `sort` method are not without their own cost. As we will see in Chapter 5, sorting is typically either $O(n^2)$ or $O(n \log n)$, so the sorting operations dominate the iteration. In the end, this algorithm will have the same order of magnitude as that of the sorting process.

2.4.3 - Anagram Detection Solution 3: Brute Force

A **brute force** technique for solving a problem typically tries to exhaust all possibilities. For the anagram detection problem, we can simply generate a list of all possible strings using the characters from `s1` and then see if `s2` occurs. However, there is a problem with this approach. When generating all possible strings from `s1`, there are n possible first characters, $n - 1$ possible characters for the second position, $n - 2$ for the third, and so on. The total number of candidate strings is $n \cdot (n - 1) \cdot (n - 2) \dots \cdot 3 \cdot 2 \cdot 1$, which is $n!$. Although some of the strings may be duplicates, the program cannot know this ahead of time and so it will still generate $n!$ different strings.

It turns out that $n!$ grows even faster than 2^n as n gets large. In fact, if `s1` were 20 characters long, there would be $20! = 2,432,902,008,176,640,000$ possible candidate strings. If we processed one possibility every second, it would still take us 77,146,816,596 years to go through the entire list. This is probably not going to be a good solution.

2.4.4 - Anagram Detection Solution 4: Count and Compare

Our final solution to the anagram problem takes advantage of the fact that any two anagrams will have the same number of a's, the same number of b's, the same number of c's, and so on. In order to decide whether two strings are anagrams, we will first count the number of times each character occurs. Since there are 26 possible characters, we can use an array of 26 counters, one for each possible character. Each time we see a particular character, we will increment the counter at that position. In the end, if the two lists of counters are identical, the

strings must be anagrams. We will take advantage of the fact that char variables in Java are stored as integers, and Java allows us to evaluate an expression like `'c' - 'a'`, which evaluates to 2. [Listing 2.4.3](#) shows this solution.

Listing 2.4.3.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Arrays;
2
3 public class Anagrams4 {
4
5     public static boolean anagramSolution4(String s1, String
6         int[] count1 = new int[26]; // initialized to all zero
7         int[] count2 = new int[26];
8
9         for (int i = 0; i < s1.length(); i++) {
10             int index = s1.charAt(i) - 'a';
11             count1[index] = count1[index] + 1;
12         }
13
14         for (int i = 0; i < s2.length(); i++) {
15             int index = s2.charAt(i) - 'a';
16             count2[index] = count2[index] + 1;
17         }
18
19         int j = 0;
20         boolean isAnagram = true;
21         while (j < 26 && isAnagram) {
22             if (count1[j] == count2[j]) {
23                 j = j + 1;
24             } else {
25                 isAnagram = false;
26             }
27         }
28         return isAnagram;
29     }
30 }
```

Again, the solution has a number of iterations. However, unlike the first solution, none of them are nested. The first two iterations used to count the characters are both based on n . The third iteration, comparing the two lists of counts, always takes 26 steps since there are 26 possible characters in the strings. Adding it all up gives us $T(n) = 2n + 26$ steps. That is $O(n)$. We have found a linear order of magnitude algorithm for solving this problem.

Before leaving this example, we need to say something about space requirements. Although the last solution was able to run in linear time, it could only do so by using additional storage to keep the two lists of character counts. In other words, this algorithm sacrificed space in order to gain time.

This is a common occurrence. On many occasions you will need to make decisions between time and space trade-offs. In this case, the amount of extra space is not significant. However, if the underlying alphabet had millions of characters, there would be more concern. As a computer scientist, when given a choice of algorithms, it will be up to you to determine the best use of computing resources given a particular problem.



Self Check

1.

Q-4: Given the following code fragment, what is its Big O running time?
(Presume that variable n has been declared elsewhere in the program.)

```
int test = 0;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        test = test + i * j;
    }
}
```

Choose one

- ☐ A. $O(n)$
- ☒ B. $O(n^2)$
- ☐ C. $O(\log n)$
- ☐ D. $O(n^3)$

Check Me

Compare me

2.

Q-5: Given the following code fragment what is its Big O running time?
(Presume that n has been declared elsewhere in the program.)

```
int test = 0;
for (int i = 0; i < n; i++) {
    test = test + i;
}

for (int j = 0; j < n; j++) {
    test = test - 1;
}
```

Choose one

- ☒ A. $O(n)$
- ☐ B. $O(n^2)$
- ☐ C. $O(\log n)$
- ☐ D. $O(n^3)$

[Check Me](#)[Compare me](#)

3.

Q-6: Given the following code fragment what is its Big O running time?
(Presume n has been declared elsewhere in the program.)

```
int i = n;
while (i > 0) {
    int k = 2 + 2;
    i = i / 2;
}
```

Choose one

- ☐ A. $O(n)$
- ☐ B. $O(n^2)$
- ☒ C. $O(\log n)$
- ☐ D. $O(n^3)$

Check Me

Compare me

You have attempted 4 of 7 activities on this page.

2.6 - ArrayLists

The designers of Java had many choices to make when they implemented the ArrayList data structure. Each of these choices could have an impact on how fast list operations perform. To help them make the right choices they looked at the ways that people would most commonly use the ArrayList (which we shall call “list” for short) data structure, and they optimized their implementation of a list so that the most common operations were very fast. Of course they also tried to make the less common operations fast, but when a trade-off had to be made the performance of a less common operation was often sacrificed in favor of the more common operation.

Two common operations are indexing and assigning to an index position. Both of these operations take the same amount of time no matter how large the list becomes. When an operation like this is independent of the size of the list, it is $O(1)$.

Another very common programming task is to grow an ArrayList by using the `add` method, which is $O(n)$, but in a very special way that we will examine in **Section 8.2**.

Here is a table of common ArrayList operations and their Big O efficiency:

Table 2.6.1.

Operation	Big O Efficiency
<code>get(index)</code>	$O(1)$
<code>set(index)</code>	$O(1)$
<code>add n items</code>	$O(n)$
<code>remove(index)</code>	$O(n)$
<code>indexOf()</code>	$O(n)$

Let’s examine the `remove` method a bit more closely. The $O(n)$ is a *worst-case* number. When we remove the first number in an ArrayList, all the remaining numbers have to be moved left one position. If we remove the last number in an ArrayList, that is a *best-case* situation, and is $O(1)$, as no elements need to be moved.

Here's the results of a program that removes the first item from an ArrayList 1000 times and compares that time to the time required to remove the last item 1000 times.

```
2,000,000 items
Remove first time: 0.3152410 sec
Remove last time: 0.0000029 sec

1,000,000 items
Remove first time: 0.0858035 sec
Remove last time: 0.0000022 sec

100,000 items
Remove first time: 0.0055089 sec
Remove last time: 0.0000017 sec
```

As you see, the worst case time is proportional to the size of the list; the best case time is relatively unchanged.

[Listing 2.6.2](#) is the program that we used, and there are some important things to note:

Listing 2.6.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.Scanner;
2 import java.util.ArrayList;
3
4 public class RemoveItem {
5
6
7     public static double timeRemove(int len, boolean removeF:
8         final int N_TRIALS = 25;
9         final int N_IGNORE = 20; // let system stabilize
10        long totalTime = 0;
11
12        ArrayList<Integer> list = new ArrayList<>(len);
13
14        for (int trial = 0; trial < N_TRIALS; trial++) {
15            int start = list.size();
16            for (int i = start; i < len; i++) {
17                list.add(i);
18            }
19            System.gc();
20
21            long startTime = System.nanoTime();
22            for (int count = 0; count < 1000; count++) {
23                if (removeFirst) {
24                    list.remove(0);
25                } else {
26                    list.remove(list.size() - 1);
27                }
28            }
29
30            if (trial >= N_IGNORE) {
```

In lines 8 and 9, we will run each test 25 times and average only the last 5 (in line 35), giving the cache time to stabilize as we saw in [Listing 2.2.3](#).

We allocate the ArrayList once before the trials, in line 12. This will be an empty list, so we have to fill it in lines 15-18. This has to be done at every trial, because each trial removes 1000 items.

Line 19 introduces a very important concept. When Java allocates and re-allocates memory, it is possible to have an area of memory that is no longer referred to. The Java Virtual Machine periodically goes through memory and collects these unused areas so they can be re-used. This process is called **garbage collection**, and it can take a fair amount of time. We don't want this to happen while we are in our timing loop, so we call `System.gc()` to explicitly invoke garbage collection. This is not a sure-fire cure; the documentation for `gc` says: "There is no guarantee that this effort will recycle any particular number of unused objects, reclaim any particular amount of space, or complete at any particular time, if at all, before the method returns or ever." Nonetheless, calling `System.gc()` is our best effort to avoid garbage collection at inopportune times.

Finally, because the inner loop repeats a thousand times, it is also important to point out that the list is decreasing in size by one each time through the loop. But even in the shortest list of 100,000 elements, this only reduces the overall size by 1% (and only 0.05% in the case of the 2,000,000 element list).

You have attempted 1 of 2 activities on this page.



New: MCP for Claude & Cursor. Your AI gets the error, trace, deploy

and code. No vibe-
debugging.

www.appsignal.com

Ads by EthicalAds

2.7 - HashMaps

The second major Java data structure is the HashMap. As you probably recall, HashMaps differ from ArrayLists in that you can access items in a HashMap (also called a “map” for short) by a key rather than a position. Later in this book you will see that there are many ways to implement a HashMap. The thing that is most important to notice right now is that the `get` and `put` methods on a HashMap are $O(1)$. Another important HashMap operation is the `containsKey` operation. Checking to see whether a key is in the HashMap or not is also $O(1)$. The efficiency of common HashMap operations is summarized in [Table 2.7.1](#). One important side note on dictionary performance is that the efficiencies we provide in the table are for average performance. In some rare cases the `contains`, `get`, and `set` operations can degenerate into $O(n)$ performance, but we will get into that in Chapter 8 when we talk about the different ways that a dictionary could be implemented.

Table 2.7.1. Big O Efficiency of Java HashMap Operations

Operation	Big O Efficiency
<code>putAll</code> (copy to another HashMap)	$O(n)$
<code>get</code>	$O(1)$
<code>put</code> (set item)	$O(1)$
<code>remove</code>	$O(1)$
<code>containsKey</code>	$O(1)$
iteration	$O(n)$

For our last performance experiment, we will compare the performance of the `contains` method for ArrayLists and the `containsKey` operation for HashMaps. In the process we will confirm that the operation for ArrayLists is $O(n)$ and the operation for HashMaps is $O(1)$. The experiment we will use to compare the two is as follows: we’ll make a list with a range of numbers in it, then we will pick numbers at random and check to see if the numbers are in the list. If our performance tables are correct, the bigger the list, the longer it should take to determine if any one number is contained in the list.

We will repeat the same experiment for a HashMap that contains numbers as the keys. In this experiment we should see that determining whether or not a number is in the HashMap is not only much faster, but the time it takes to check should remain constant even as the HashMap grows larger.

[Listing 2.7.2](#) implements this comparison. Notice that we are performing exactly the same operation, even though they have different names:

`list.contains(number)` and `map.containsKey(number)`.

Listing 2.7.2.

Save & Run



Original - 1 of 1

Show CodeLens

Reformat

```
1 import java.util.ArrayList;
2 import java.util.HashMap;
3 import java.util.Random;
4
5 public class ListVsMap {
6
7     public static void main(String[] args) {
8
9         System.out.println("size      list      map");
10        for (int size = 10_000; size < 1_000_001; size += 20_000) {
11            ArrayList<Integer> list = new ArrayList<>(size);
12            HashMap<Integer, Integer> map = new HashMap<>();
13
14            for (int key = 0; key < size; key++) {
15                list.add(key);
16                map.put(key, key); // key and value are the same
17            }
18            System.gc();
19            Random generator = new Random();
20
21            long startListTime = System.nanoTime();
22            for (int i = 0; i < 1000; i++) {
23                int lookFor = generator.nextInt(size);
24                boolean found = list.contains(lookFor);
25            }
26            long elapsedListTime = System.nanoTime() - startListTime;
27
28            long startMapTime = System.nanoTime();
29            for (int i = 0; i < 1000; i++) {
30                Integer lookFor = generator.nextInt(size);
```

Figure 2.7.3 summarizes the results of running Listing 2.7.2. You can see that the dictionary is consistently faster. For the smallest size of 10,000 elements, a HashMap is 58.2 times faster than an ArrayList. For the largest size of 990,000 elements the HashMap is 3,972 times faster! You can also see that the time it takes for the `contains` method on the ArrayList grows linearly with the size of the list. This verifies the assertion that the `contains` method on a list is $O(n)$. It can also be seen that the time for the `containsKey` method on a HashMap is constant even as the dictionary size grows.

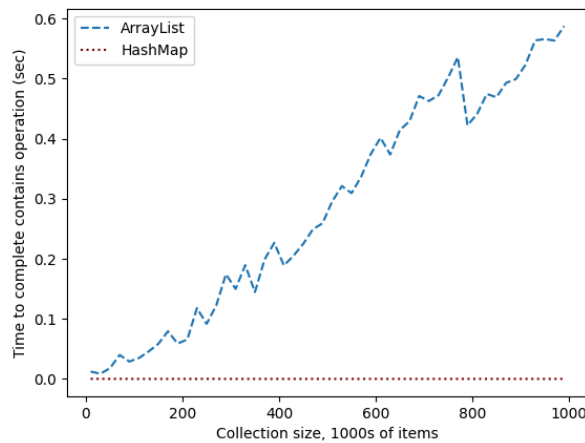


Figure 2.7.3. Comparing Time to Find Elements for Java ArrayList and HashMap

Self Check

1.

Q-1: Which of the ArrayList operations shown below is not $O(1)$?

Choose one

- ☒ A. `myList.remove(0)`
- ☐ B. `myList.remove(myList.size() - 1)`
- ☐ C. `myList.add()`
- ☐ D. `myList.get(10)`
- ☐ E. all of the above are $O(1)$

Check Me

Compare me

2.

Q-2: Which of the HashMap operations shown below is $O(1)$?

Choose one

- ☐ A. `myMap.containsKey("x")`
- ☐ B. `myMap.remove("x")`
- ☐ C. `myMap.put("x", 22)`
- ☒ D. all of the above are $O(1)$

Check Me

Compare me

You have attempted 3 of 4 activities on this page.

EthicalAds

ADS BY ETHICAL



Ultimate BIN Database

Fresh BIN Database of
6-11 digits. Token
support. **Get data**

www.binbase.com

Ads by EthicalAds